

数字 1： 8bit 图像可配置预处理模块实现

【试题说明】:

根据题目要求实现一个 8bit 图像的可配置预处理 ASIC 模块。输入图像最大为 132 高 200 宽彩色图像或灰度图像，通过配置内置寄存器，完成对图像的三种处理动作和 bypass 功能：

- 1. 图像灰度化处理
- 2. 图像 RGB 色域到 HSV 色域转换处理
- 3. 图像局部二值化处理
- 4. bypass 功能，即直通，ASIC 不做任何处理

ASIC 模块结构如图 1 所示：

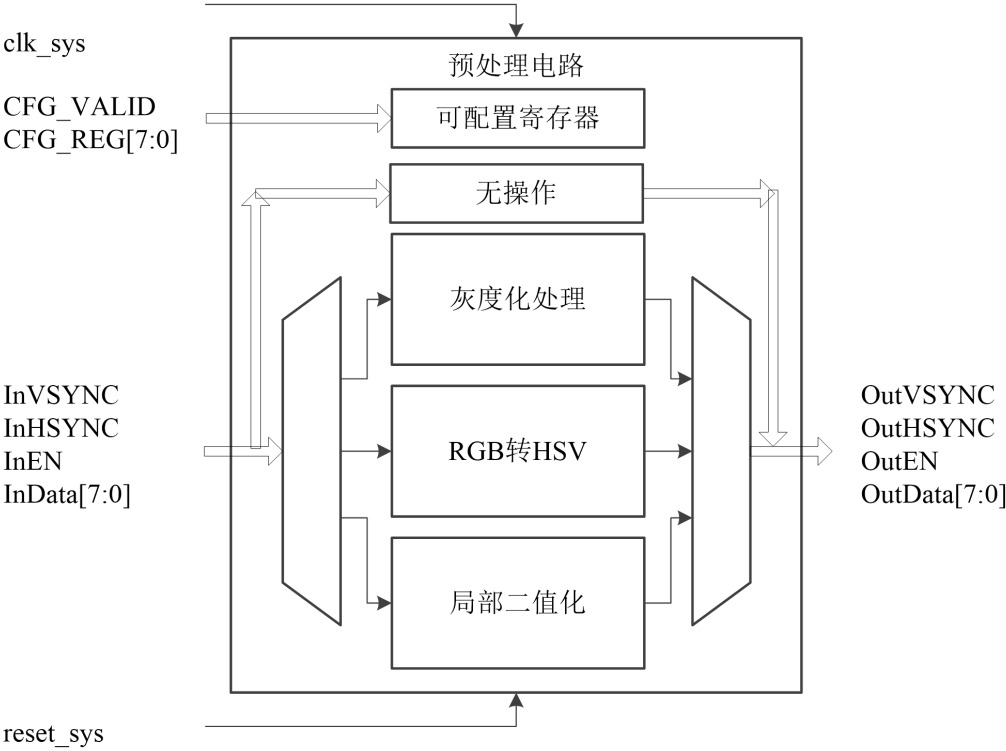


图 1 8bit 图像的可配置预处理 ASIC 模块

ASIC 内部寄存器 MODE_REG 的 8bit 功能配置如下表：

MODE_REG [1:0]	2'b00	2'b01	2'b10	2'b 11
	无操作	灰度化	RGB 转化 HSV	局部二值化

局部二值化时 MODE_REG[6:2] 配置如下表：

	MODE_REG [6]	MODE_REG [5:4]	MODE_REG [3:2]
当 MODE_REG [1:0] == 2'b11 时,[6:2]为局部二值化的配置参数	0 正常，1 反色	局部窗口半宽度 W _{half}	局部窗口半高度 h _{half}

当需要完成图像灰度化处理时，配置内部寄存器 MODE_REG [1:0]= 2'b01，然后输入彩色图像（R、G、B 三通道），经过硬件计算，输出灰度图像（单通道）。

当需要完成图像 RGB 色域到 HSV 色域转换处理时，配置内部寄存器

MODE_REG [1:0]=2'b10, 然后输入彩色图像 (R、G、B 三通道), 经过硬件计算, 输出 HSV 图像 (H、S、V 三通道)。

当需要完成局部二值化处理时, 配置内部寄存器 MODE_REG [1:0]=2'b11, 然后根据需求配置局部半宽度和局部半高度, 以及二值化模式 (正常或反色)。输入图像为灰度图像 (单通道), 输出图像为二值图像 (单通道)。

如果内部寄存器 MODE_REG [1:0]=2'b00, 则 ASIC 无操作, 实现 bypass 功能, 输出与输入保持一致。

组委会提供模块的顶层文件 asic.v 和仿真 case。case 中有三种不同大小图片的仿真案例: min、small、large, 其图片大小分别为高 43 宽 50、高 88 宽 100、高 132 宽 200。每个 case 中提供彩色图像和灰度图像的 mem 文件, 参赛队需要根据自己的仿真工具搭建测试环境, 并生成三个案例的结果文件进行提交。

【算法说明】:

1. 灰度化处理算法说明:

$$\text{GRAY} = 0.3 \cdot r + 0.59 \cdot g + 0.11 \cdot b$$

r、g、b 分别为彩色图像的 R、G、B 三个通道值, 每个通道的范围为 0~255, R 为红色通道、G 为绿色通道、B 为蓝色通道, 通过上式可将彩色图像转化为灰度图像 GRAY, 通道的范围为 0~255。

2. RGB 转 HSV 算法说明:

算法公式如下:

$$H = \begin{cases} 0^\circ, & \text{if } \max == \min \\ 60^\circ \times \frac{g - b}{\max - \min} + 0^\circ, & \text{if } \max == r \text{ and } g \geq b \\ 60^\circ \times \frac{g - b}{\max - \min} + 360^\circ, & \text{if } \max == r \text{ and } g < b \\ 60^\circ \times \frac{b - r}{\max - \min} + 120^\circ, & \text{if } \max == g \\ 60^\circ \times \frac{r - g}{\max - \min} + 240^\circ, & \text{if } \max == b \end{cases}$$

$$S = \begin{cases} \frac{\max - \min}{\max}, & \text{if } \max \neq 0 \\ 0, & \text{if } \max = 0 \end{cases}$$

$$V = \max / 255$$

其中 r、g、b 分别为图像 RGB 三个通道的像素值, 范围 0~255, max 为 r、g、b 中最大值, min 为 r、g、b 中最小值。H 为整数, 范围 0~360, S、V 为小数, 范围 0~1。硬件设计中, H、S、V 均需量化至无符号 8bit 整型, 范围 0~255。

3. 局部二值化算法说明:

输入图像为灰度图像, 根据寄存器的配置参数确定局部二值化的大小, 配置参数为半宽度 w_{half} 和半高度 h_{half} , 局部二值化的示意图如下图 2 所示。

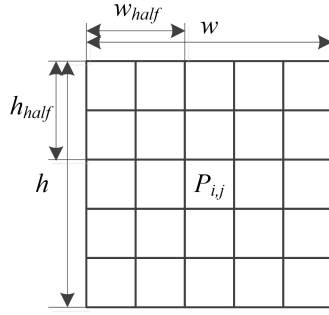


图 2 局部二值化的局部窗口示意图

局部二值化窗口的半宽度 w_{half} 和半高度 h_{half} 的最大值为 3, 则窗口最大为 7x7, 局部窗口的大小为:

$$\begin{cases} w = w_{half} * 2 + 1 \\ h = h_{half} * 2 + 1 \end{cases}$$

首先求解该区域的像素均值 AVEG:

$$AVEG = \frac{\sum_{i-w_{half}}^{i+w_{half}} \sum_{j-h_{half}}^{j+h_{half}} p_{i,j}}{w * h}$$

正常模式下局部二值化的算法公式为:

$$p'_{i,j} = \begin{cases} 0, & \text{if } AVEG > p_{i,j} \\ 255, & \text{if } AVEG \leq p_{i,j} \end{cases}$$

反色模式为

$$p'_{i,j} = \begin{cases} 255, & \text{if } AVEG > p_{i,j} \\ 0, & \text{if } AVEG \leq p_{i,j} \end{cases}$$

其中 $p_{i,j}$ 为局部窗口中心像素, $p'_{i,j}$ 为中心像素的局部二值化结果。

以 3x3 的局部窗口为例, 算法过程如图 3 所示, 不考虑图像边界, 第一个 3x3 的窗口按照公式计算后, 得到二值化图像的第一个像素。窗口按照蓝线移动, 逐步计算二值化图像的所有像素值。

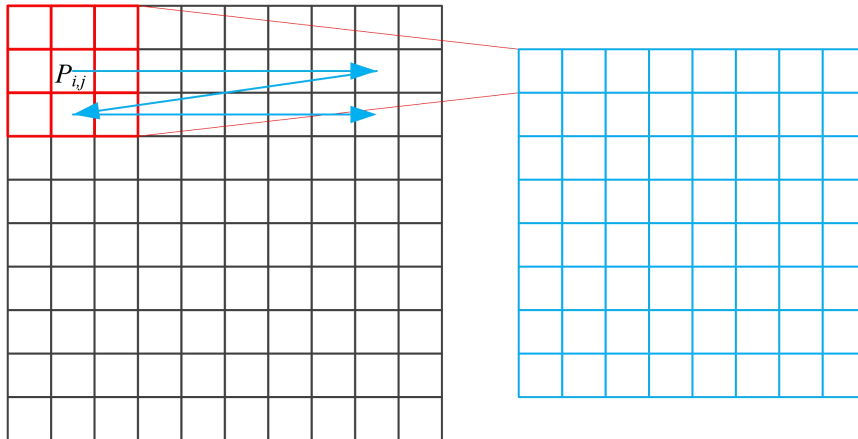


图 3 3x3 窗口局部二值化的示意图

【输入输出信号】:

信号名称	I/O	位宽	描述
clk_sys	I	1	工作时钟
reset_sys	I	1	系统异步复位信号，低电平有效
InVSYNC	I	1	帧同步脉冲信号，上升沿有效
InHSYNC	I	1	行同步脉冲信号，上升沿有效
InEN	I	1	像素数据有效标志，高电平有效
InData	I	8	输入像素数据（R,G,B）
CFG_VALID	I	1	配置寄存器写使能信号，上升沿有效
CFG_REG	I	8	输入配置寄存器
OutVSYNC	O	1	帧同步信号，上升沿有效
OutHSYNC	O	1	行同步信号，上升沿有效
OutEN	O	1	像素数据有效标志，高电平有效
OutData	O	8	输出像素数据（H,S,V）、（GRAY）（二值 0/255）

【时序要求】:

灰度图像与彩色图像的输入输出时序如图 4 所示。当帧同步信号 InVSYNC 上升沿有效时，代表图像当前一帧的结束和下一帧的开始。当行同步信号 InHSYNC 上升沿有效时，代表图像当前一行的结束和下一行的开始。当 InEN 信号有效时，InData 像素值有效。输入为灰度图像（单通道）时，每一个周期为一个像素值，当输入为彩色图像（R、G、B 三通道）时，每三个周期为一个像素值。要以 HSV 输出时，与彩色图像输入时序一致，每三个周期为一个像素。行同步信号 InHSYNC 与像素使能信号 InEN 的延迟大于等于 1 个周期。

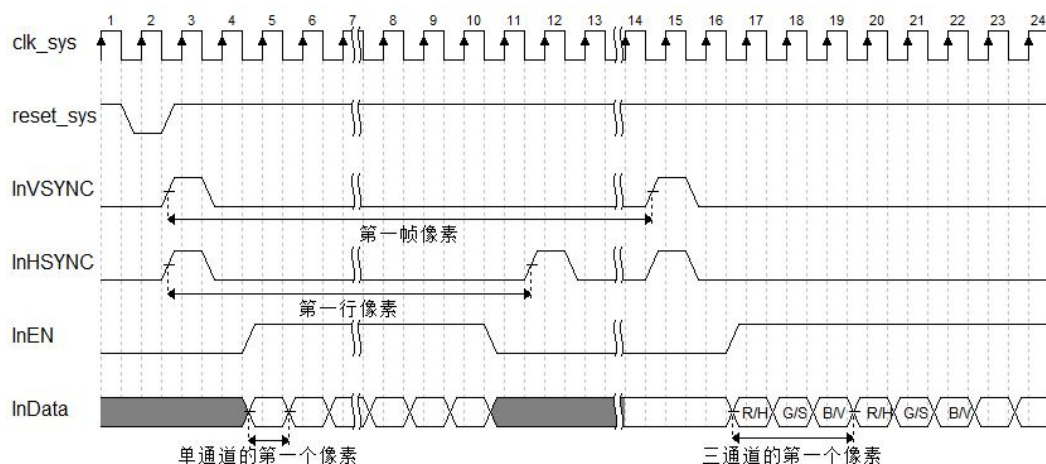


图 4 灰度图像与彩色图像的输入输出时序

可配置寄存器的时序如图 5 所示，在 CFG_VALID 上升沿输入寄存器的值 CFG_REG，下一个周期更新内置寄存器 MODE_REG 的值：

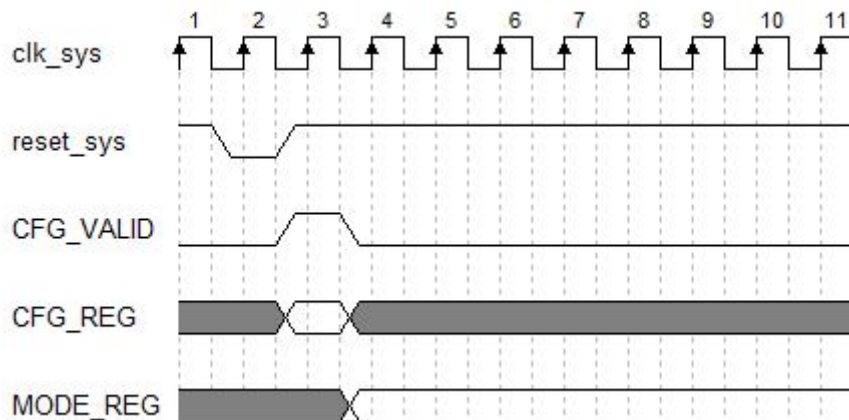


图 5 可配置寄存器的时序

【提交内容】:

参赛队按照如下表结构提交相应文件，其中 src 中为硬件设计代码，min、small、large 分别为三种 case 的仿真结果。顶层模块的端口不能改动，所有的仿真结果文件名和顶层模块文件名不能改动。

文件夹	文件夹或文件名		描述
answer	格式不限，文件或文件夹名需体现为设计文档		设计文档
	格式不限，文件或文件夹名需体现为验证报告		验证报告
	src	*.v、*sv	硬件设计代码
		asic.v	硬件设计顶层文件
	min	rgb2gray.mem	灰度化算法运行结果
		rgb2hsv.mem	rgb 转 hsv 运行结果
		binarization.mem	局部二值化运行结果，窗口大小 3x3，模式为 0
		non.mem	灰度图像的 bypass 运行结果
	small	rgb2gray.mem	灰度化算法运行结果
		rgb2hsv.mem	rgb 转 hsv 运行结果
		binarization.mem	局部二值化运行结果，窗口大小 5x5，模式为 0
		non.mem	灰度图像的 bypass 运行结果
	large	rgb2gray.mem	灰度化算法运行结果
		rgb2hsv.mem	rgb 转 hsv 运行结果
		binarization.mem	局部二值化运行结果，窗口大小 7x7，模式为 1
		non.mem	灰度图像的 bypass 运行结果

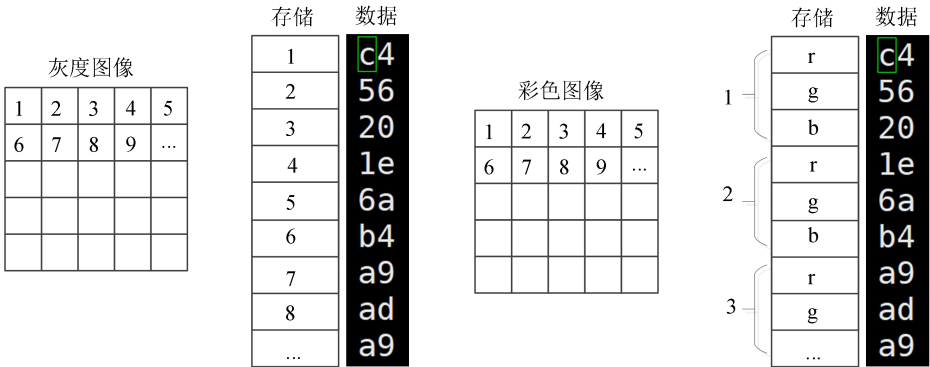
【EDA 工具】: 选择大赛官方提供的 FPGA 仿真软件或 VCS 仿真软件

【仿真 demo】:

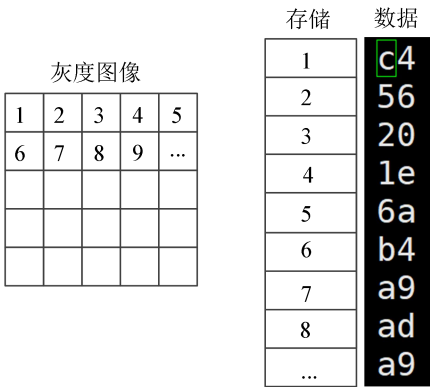
组委会提供仿真 demo 及顶层模块，其结构如下图：

```
case/  
case/asic.v  
case/large/  
case/large/gray_large.mem  
case/large/rgb_large.mem  
case/min/  
case/min/rgb_min.mem  
case/min/gray_min.mem  
case/small/  
case/small/gray_small.mem  
case/small/rgb_small.mem
```

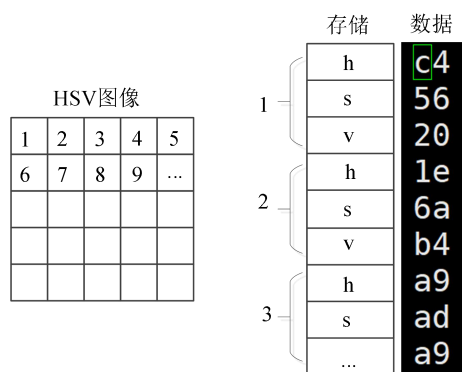
为避免参赛队仿真结果与组委会仿真结果不一致，**硬件设计顶层文件 asic.v 的文件名、模块名、输入输出信号名等不可修改**。参赛队需要验证 min、small 和 large 三个仿真案例。其中 gray_*.mem 为灰度图像，测试功能 3 和功能 4，rgb_*.mem 为彩色图像，测试功能 1 和功能 2。*.mem 文件中图像按照 Z 型存放，并且为 ASCII 码形式的十六进制数，例如下图所示：



功能 1、功能 3 和功能 4 计算结果的 mem 文件格式与 gray_*.mem 文件结构一致，如下图：



功能 2 计算结果的 mem 文件格式与 rgb_*.mem 文件结构一致，如下图：



【设计要求】:

1. 完成**设计文档**，格式不限，包括但不限于设计思路、算法优化思路、框图说明、模块电路设计等。设计文档清晰，能准确描述设计方案，准确指导代码编写。

2. 设计符合题目要求，代码实现和文档描述一致，代码结构合理，思路清晰，可读性好。提供**验证报告**，包含但不限于各功能仿真关键结果、波形截图。验证报告文档清晰，与硬件设计代码仿真结果一致，满足题目要求。

3. 硬件设计顶层文件 `asic.v` 的文件名、模块名、输入输出信号名等不可修改。参赛队需要验证 `min`、`small` 和 `large` 三个仿真案例，提供的结果文件需满足要求。组委会将根据提供的代码进行仿真测试，和参赛队提供的结果进行对比，并计算与软件结果的误差。

4. 硬件设计可使用 `verilog` 和 `system verilog` 两种硬件描述语言，不得使用不可综合语句。顶层模块中建议不使用 `parameter` 等传参语句，如若使用，默认使用初始值。由于传参导致的结果不一致问题，由参赛队自己负责。

5. 评判标准:

	设计项目	分值	描述
文档	设计文档、代码风格和验证报告	30	各文档结构清晰，能准确描述设计方案、验证结果。可准确指导代码编写，代码结构合理，思路清晰，可读性好。验证报告中可通过波形反映设计思路。
完成度	RGB 转灰度	5	功能完成度，并根据误差大小给分
	RGB 转 HSV	15	功能完成度，并根据误差大小给分
	局部二值化	20	功能完成度，并根据误差大小给分
	功能完成性	5	可通过配置寄存器实现三种功能和 <code>bypass</code> 的切换
优化设计	硬件算法优化	25	对影响设计面积和工作频率的算法关键点在设计文档和验证报告中 进行设计和验证 ，并在满足精度的情况下，优化硬件算法，减小硬件面积或提高工作频率。